

Spotify Challenge V 1.0

Autor - Sebastian Barcovsci || Última edición del documento: Enero de 2025

Acerca del Proyecto

Una aplicación web que permite a los usuarios buscar canciones utilizando la API de Spotify, pero además implementa un sistema de autenticación de usuarios mediante JWT (JSON Web Token).

Los usuarios pueden registrarse, iniciar sesión, buscar y guardar canciones en una lista de favoritos.

Puntos destacables:

- Utiliza la API de Spotify utilizando el método de autorización de [Clients Credentials](#). De esta manera, la API Rest desarrollada se encarga de encriptar el ID y la llave secreta de la **API de Spotify** y solicitar el token de acceso, que luego se proporcionará al usuario.
- Implementa un sistema de autenticación de usuario utilizando JWT. El token se genera cuando un usuario inicia sesión correctamente a través del **cliente desarrollado en Angular** (front), y este se solicitará para acceder a las funcionalidades protegidas de la aplicación.
- Persiste la información del usuario (nombre de usuario, contraseña encriptada y roles), así como también la información de sus canciones favoritas vinculadas (nombre de la canción, artista, álbum, etc).

Navegación a los puntos destacados:

- ➔ [Creando una app de Spotify para desarrolladores](#)
- ➔ [Reemplazando Client Secret y Client ID](#)
- ➔ [Endpoints de la API Rest, respuestas y autenticación](#)
- ➔ [Ejemplo de un request a la API de Spotify](#)

Base de datos, Spotify Dev y configuraciones previas

En la carpeta *database* del repositorio se encuentra la base de datos diseñada para brindar los servicios de persistencia de la API. A continuación se muestra a modo de ejemplo cómo importar el script hacia *MySQL Workbench*.

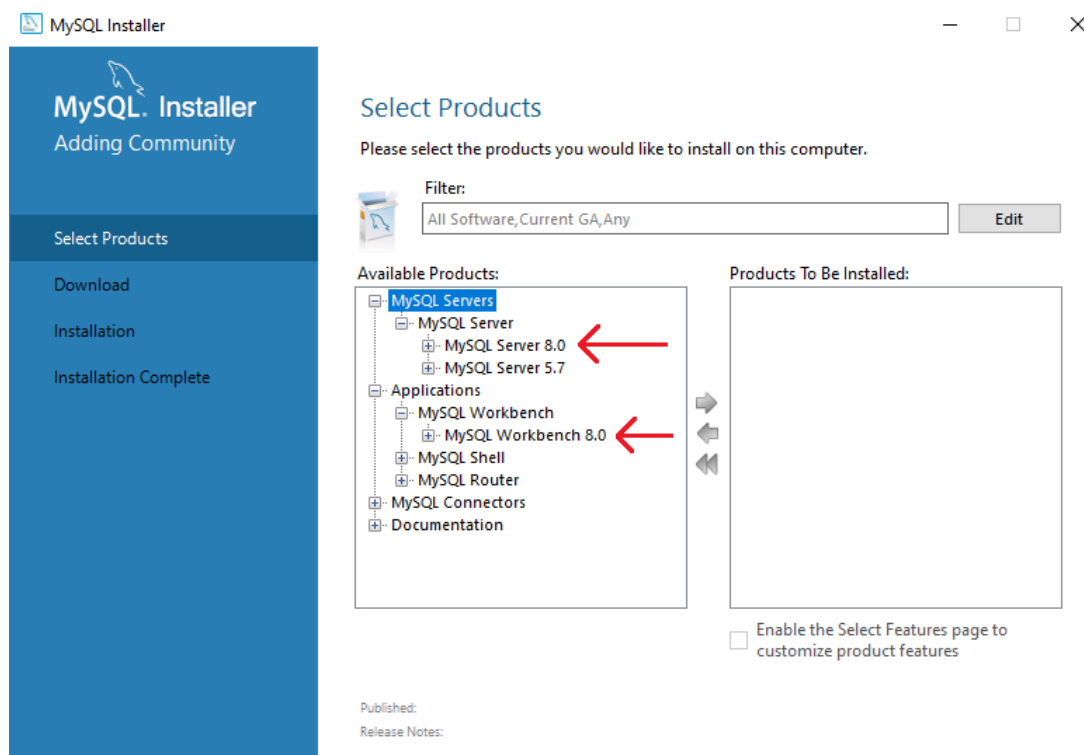
Instalar MySQL Server y MySQL Workbench

En este paso se detalla las herramientas necesarias para operar con la base de datos. Se puede omitir si ya se cuenta con ellas.

Para empezar, será necesario descargar el instalador de MySQL, accediendo a este enlace, seleccionando *MySQL Installer for Windows*:

<https://dev.mysql.com/downloads/installer/>

Una vez ejecutado el instalador, seleccionar la opción *Custom* y añadir *MySQL Server* y *MySQL Workbench*:

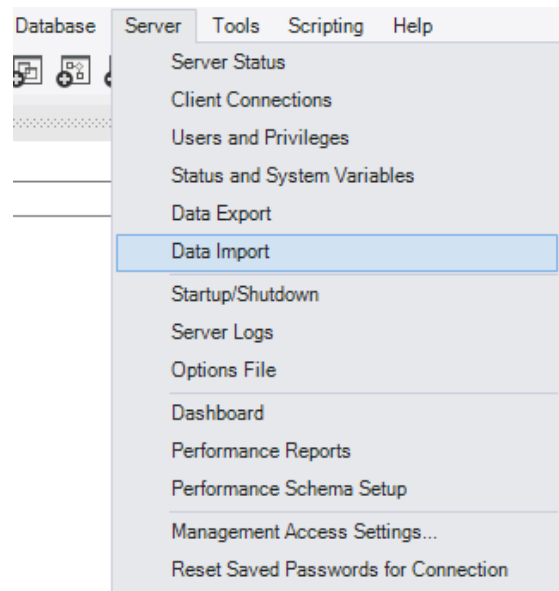


Importante: durante la instalación, se solicitará crear una contraseña para el *root*, así como seleccionar un puerto para MySQL servers (por defecto 3306). Ambos datos serán relevantes para la configuración previa de la API.

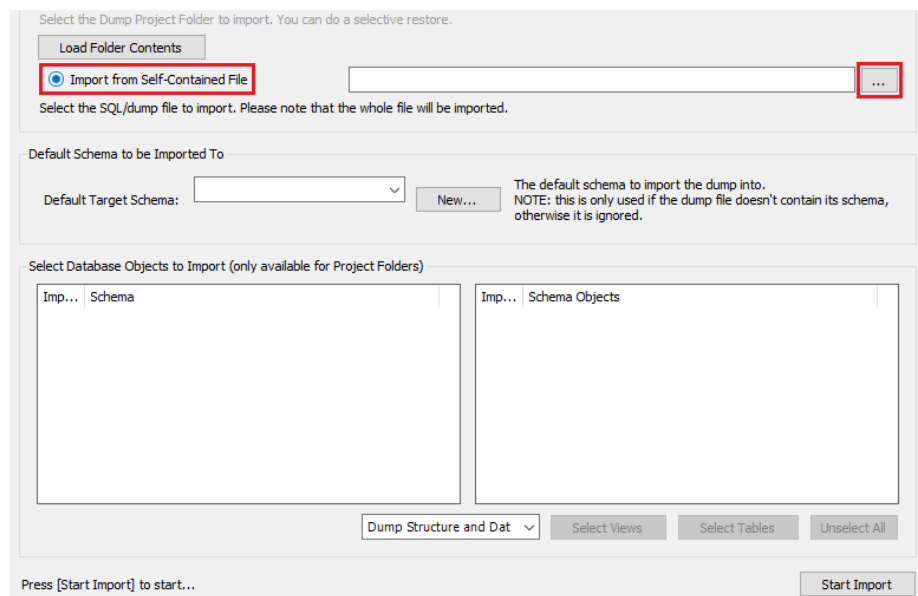
Importando la base de datos a la instancia local

Debemos contar con el archivo *db_spotify_api.sql* en la carpeta *database* del repositorio y ejecutar MySQL Workbench.

Una vez creada una instancia local, dirigirse a *Server > Data Import*.



Allí seleccionamos la opción *Import from Self-Contained File* y buscamos el archivo descargado.



Por último, continuar con *Start Import* para generar el schema.

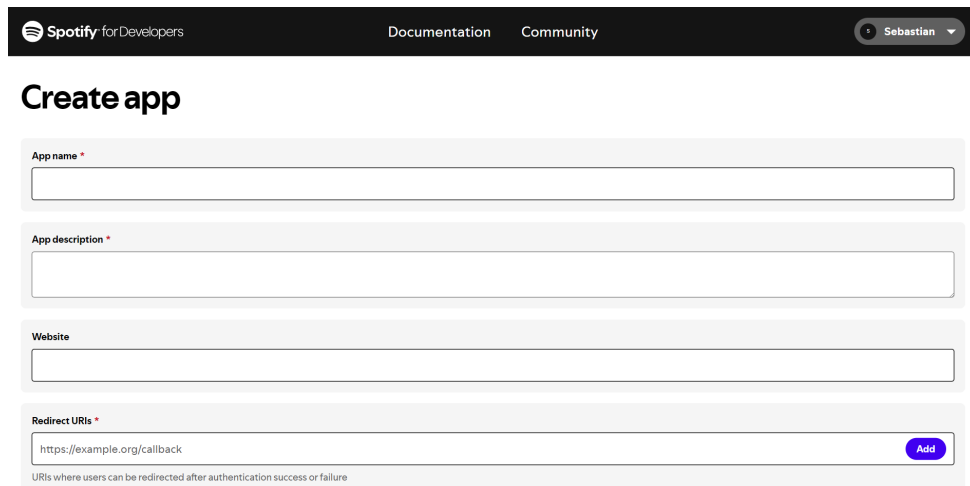
Con esto, la base de datos queda lista para acceder a través del puerto seleccionado durante la instalación.

Creando una app de Spotify para desarrolladores

Crearemos un app de Spotify para solicitar el token de acceso a su API y poder consumir los distintos endpoints que nos brinda. Para eso debemos acceder al siguiente enlace y crear un usuario:

<https://developer.spotify.com/>

Luego de iniciar sesión , nos dirigimos al dashboard y creamos la app:

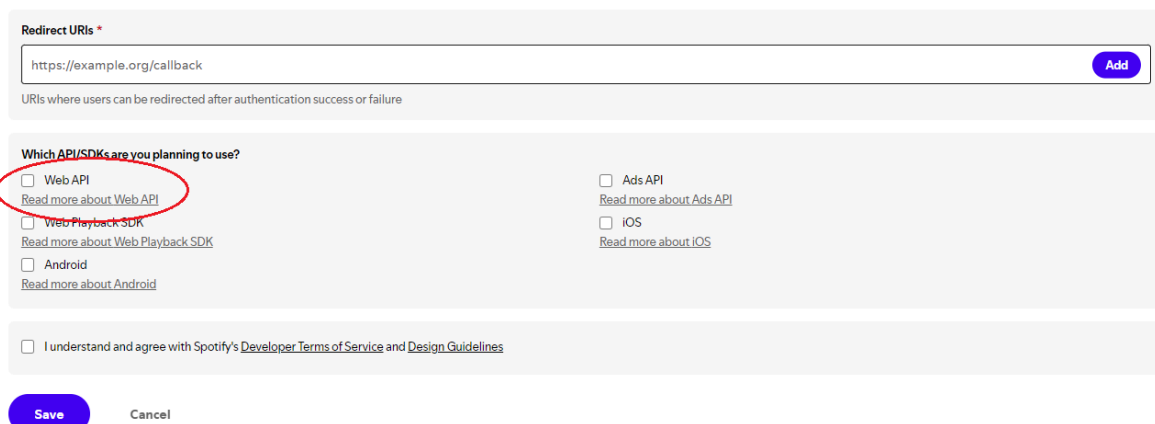


The screenshot shows the 'Create app' form on the Spotify Developer Dashboard. The form has a dark header with the Spotify logo, 'for Developers', and links to 'Documentation' and 'Community'. A user profile 'Sebastian' is visible in the top right. The form fields are: 'App name' (empty), 'App description' (empty), 'Website' (empty), and 'Redirect URIs' (containing 'https://example.org/callback' with an 'Add' button). Below the 'Redirect URIs' field is a small text note: 'URIs where users can be redirected after authentication success or failure'.

Le damos un nombre y una descripción, y en la casilla de *Redirect URIs* colocamos el puerto por defecto de Angular o Spring (*).

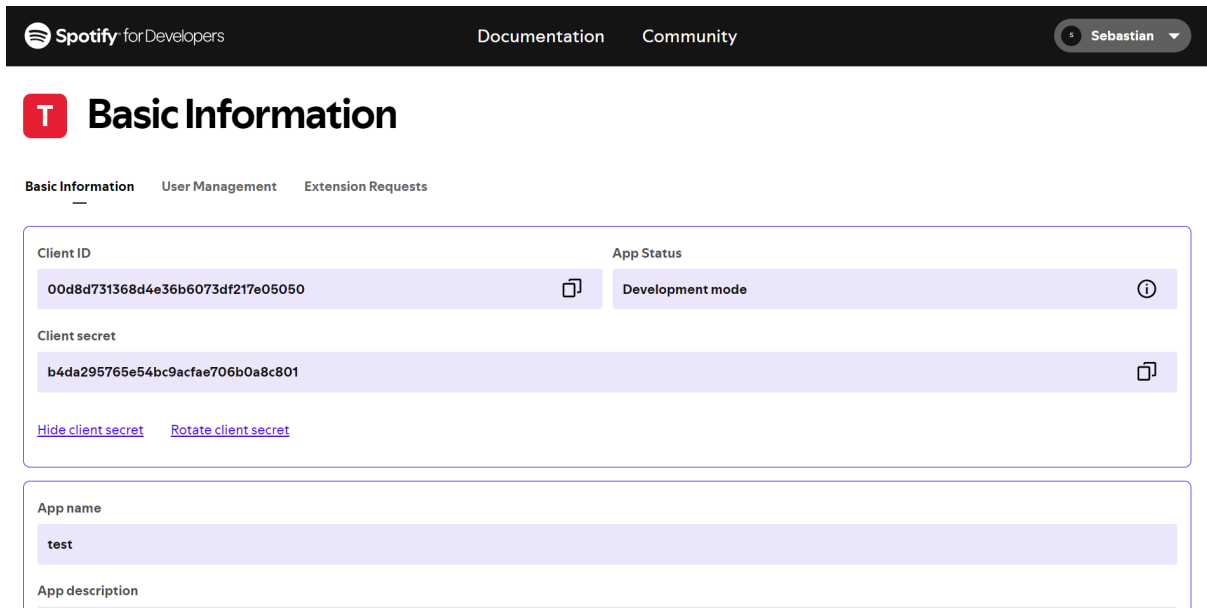
***Aclaración:** Debido a como está planteado el proyecto, no se hace uso de esta característica, por lo que colocamos un redireccionamiento provisorio para completar el campo obligatorio (ej: `http://localhost:4200`). En futuras actualizaciones del proyecto, la responsabilidad de solicitar el token será dada al cliente del front, para aprovechar esta característica en el back, como por ejemplo, para desarrollar métodos destinados a métricas.

Por último, marcamos la casilla de API Web y guardamos.



The screenshot shows the 'Which API/SDKs are you planning to use?' section of the Spotify Developer Dashboard. It features a list of checkboxes for different APIs and SDKs: 'Web API' (checked), 'Web Playback SDK', 'Android', 'Ads API', 'iOS', and 'Read more about Web API', 'Read more about Ads API', 'Read more about Web Playback SDK', 'Read more about iOS', and 'Read more about Android'. Below this list is a checkbox for 'I understand and agree with Spotify's Developer Terms of Service and Design Guidelines'. At the bottom are 'Save' and 'Cancel' buttons.

Ahora, desde el dashboard ingresamos a la app recién creada y nos dirigimos a *Basic Information*:

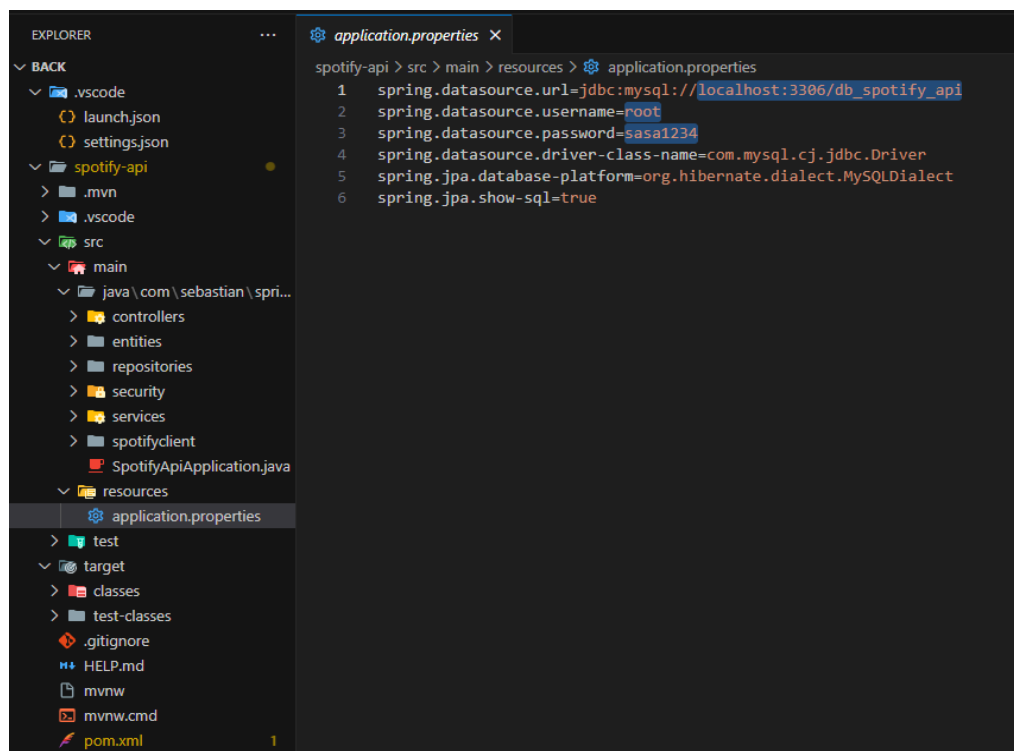


The screenshot shows the Spotify for Developers dashboard. At the top, there's a navigation bar with the Spotify logo, 'for Developers', 'Documentation', 'Community', and a user profile 'Sebastian'. Below this, the 'Basic Information' section is active, showing fields for Client ID (00d8d731368d4e36b6073df217e05050), Client secret (b4da295765e54bc9acfae706b0a8c801), App name (test), and App description. There are also links for 'Hide client secret' and 'Rotate client secret'. The App Status is set to 'Development mode'.

Aquí tendremos acceso al **Client ID** y al **Client Secret** para solicitar el token de autorización que requiere la API de Spotify para hacer uso de sus endpoints.

Configurando la API

A continuación, configuraremos el acceso a la base de datos dentro de la API Rest, para eso nos dirigiremos a *resources/application.properties*, dentro de la carpeta del proyecto, en el back:



The screenshot shows a VS Code editor with the Explorer on the left and the application.properties file open in the main editor. The Explorer shows the project structure with folders like .vscode, launch.json, settings.json, .mvn, .vscode, src, main, java, controllers, entities, repositories, security, services, spotifyclient, SpotifyApiApplication.java, resources, test, target, classes, test-classes, .gitignore, HELP.md, mvnw, mvnw.cmd, and pom.xml. The application.properties file contains the following configuration:

```
spotify-api > src > main > resources > application.properties
1 spring.datasource.url=jdbc:mysql://localhost:3306/db_spotify_api
2 spring.datasource.username=root
3 spring.datasource.password=sasa1234
4 spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
5 spring.jpa.database-platform=org.hibernate.dialect.MySQLDialect
6 spring.jpa.show-sql=true
```

Modificaremos las líneas resaltadas en la imagen, en la primera colocaremos la ruta hacia el **puerto de MySQL Server**, si se dejó por defecto durante la instalación, no será necesario modificar esta línea.

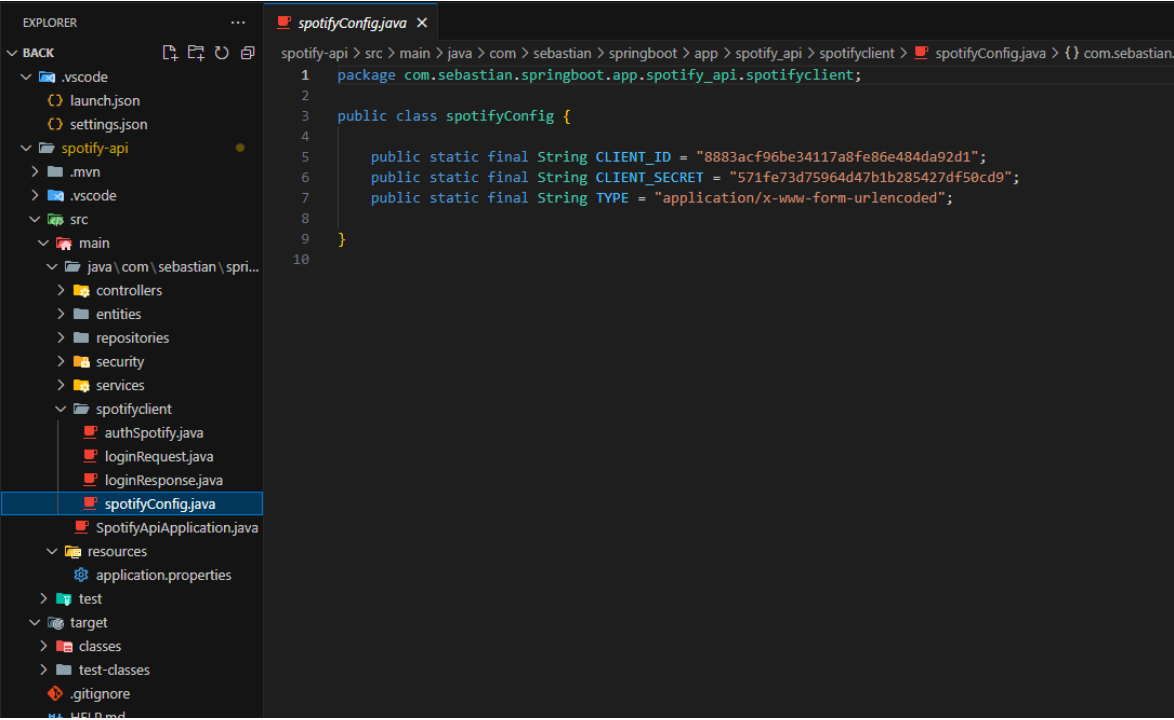
En la segunda línea, colocamos el username de MySQL, por defecto **root**.

Por último, en la tercera línea, colocamos la contraseña seleccionada durante la instalación de MySQL Server.

De esta manera, la API quedará configurada y podrá comunicarse con la base de datos, permitiendo realizar operaciones de persistencia y consulta.

Reemplazando Client Secret y Client ID

Finalmente, solo queda reemplazar las credenciales que obtuvimos al crear la [app de Spotify](#). Para eso nos dirigiremos a la clase *spotifyConfig.java*, dentro de la carpeta *spotifyClient*, en el back:



```
1 package com.sebastian.springboot.app.spotify_api.spotifyclient;
2
3
4 public class spotifyConfig {
5
6     public static final String CLIENT_ID = "8883acf96be34117a8fe86e484da92d1";
7     public static final String CLIENT_SECRET = "571fe73d75964d47b1b285427df50cd9";
8     public static final String TYPE = "application/x-www-form-urlencoded";
9 }
10
```

Allí reemplazamos el valor de CLIENT_ID y CLIENT_SECRET, credenciales correspondientes.

Con esto, la API está lista para ser ejecutada, a continuación se enumeran y describen los distintos endpoints a los que se pueden acceder.

Endpoints de la API Rest, respuestas y autenticación

Los endpoints que se listan a continuación están ordenados según el rol requerido para consumirlos, en orden jerárquico.

URL Base: localhost:8080

Post /api/users/register

|| Registra un nuevo usuario.

Rol requerido: ninguno

Parámetros requeridos:

- Ninguno

Request Body:

```
{
  "username": "nuevo user",
  "password": "12345"
}
```

Response Body:

```
{
  "id": 4,
  "username": "nuevo user",
  "roles": [
    {
      "id": 2,
      "name": "ROLE_USER"
    }
  ],
  "enabled": true
}
```

Post /login

|| Inicia sesión con un usuario registrado.

Rol requerido: ninguno

Parámetros requeridos:

- Ninguno

Request Body:

```
{
  "username": "nuevo user",
  "password": "12345"
}
```

Response Body:

```
{
  "token": (eyJhbGciOiJI... ),
  "username": "nuevo user"
}
```

Get /api/users/spotify/gettoken

|| Solicita el token de autorización de la API de Spotify.

Rol requerido: USER

Parámetros requeridos: Header

```
{
  "Content-Type": "application/json",
  "Authorization": "Bearer " + token de JWT
}
```

Request Body:

Response Body:

```
{
  "token": (eyJhbGciOiJI ... )
}
```

Get /api/tracks/{username}

|| Devuelve la lista de canciones favoritas del usuario solicitado.

Rol requerido: USER

Parámetros requeridos: Header

```
{
  "Content-Type": "application/json",
  "Authorization": "Bearer " + token de JWT
}
```

Request Body:

Response Body:

```
[
  {
    "id": 1,
    "name": "Mujer Amante",
    "artist": "Rata Blanca",
    "album": "(Col 2x1 ...)",
    "spotifyid": "(spotify:t...)",
    "imgurl": "(https://i.sc...)",
    "username": "seba"
  }
  { ... }
]
```


Post /api/tracks/save/{username}

|| Persiste una canción en la BD, vinculando al usuario, y retorna la

Rol requerido: USER

Parámetros requeridos: Header

```
{
  "Content-Type": "application/json",
  "Authorization": "Bearer " + token de JWT
}
```

Request Body:

```
[
  {
    "id": 1,
    "name": "Mujer Amante",
    "artist": "Rata Blanca",
    "album": "(Col 2x1 ...)",
    "spotifyid": "(spotify:t...)",
    "imgurl": "(https://i.sc...)",
    "username": "seba"
  }
]
```

Response Body:

```
[
  {
    "id": 1,
    "name": "Mujer Amante",
    "artist": "Rata Blanca",
    "album": "(Col 2x1 ...)",
    "spotifyid": "(spotify:t...)",
    "imgurl": "(https://i.sc...)",
    "username": "seba"
  }
  { ... }
]
```

Get /api/users

|| Devuelve lista de usuarios registrados.

Rol requerido: USER

Rol requerido: ADMIN

Parámetros requeridos: Header

```
{
  "Content-Type": "application/json",
  "Authorization": "Bearer " + token de JWT
}
```

Request Body:

Response Body:

```
[
  {
    "id": 4,
    "username": "nuevo user",
    "roles": [ ... ],
    "enabled": true
  }
  { ... }
]
```

Post /api/users/create

|| Registra un nuevo Usuario o Admin.

Rol requerido: USER

Rol requerido: ADMIN

Parámetros requeridos: Header

```
{
  "Content-Type": "application/json",
  "Authorization": "Bearer " + token de JWT
}
```

Request Body:

```
{
  "username": "nuevo user",
  "password": "12345"
}
```

Response Body:

```
{
  "id": 4,
  "username": "nuevo user",
  "roles": [
    {
      "id": 2,
      "name": "ROLE_USER"
    },
    {
      "id": 1,
      "name": "ROLE_ADMIN"
    }
  ],
  "enabled": true
}
```

Ejemplo de un request a la API de Spotify

En la carpeta de front se encuentra un ejemplo simple de un cliente creado en Angular. La finalidad de este ejemplo es mostrar cómo consumir tanto la API desarrollada en Spring como la API de Spotify.

A continuación se mostrará un ejemplo de cómo realizar una petición en la API de Spotify, utilizando el [token de autorización generado en el back](#).

Get

`https://api.spotify.com/v1/search?q={trackName}&type={type}&limit={limit}`

|| Devuelve una o varias canciones que cumplan con el criterio.

Parámetros requeridos:

- Header
 - {
 "Authorization": "Bearer " + token de Spotify
}
- {trackName} - Nombre de la canción
- {type} - Tipo de ítem a buscar (en este caso 'track')
- {limit} - límite máximo de resultados esperados en el response

Ejemplo:

`https://api.spotify.com/v1/search?q=rata+blanca&type=track&limit=1`

Elementos relevantes del response:

`track.name
track.uri
track.album.images[0].url
track.artists[0].name
track.album.name`

Debido al tamaño del Json que se recibe en el response, en el ejemplo anterior se muestra como navegar hacia los datos que son necesarios para el método POST de nuestro Back.

Para una mejor comprensión, se recomienda realizar una prueba utilizando el simulador en el siguiente [enlace](#).

Allí se podrá observar el Json recibido con mayor detalle, y recibir una descripción de los datos obtenidos.